# Urna Aberta: An Offline Ballot Protocol with Publicly Checkable Evidence

Urna Aberta contributors

Educational research prototype; no institutional affiliation claimed

**Abstract.** Electronic election systems face a difficult boundary: a voter needs a private, usable interaction, while observers need evidence that reported totals came from valid ballots and were not silently altered or omitted. This paper presents Urna Aberta, an educational prototype that connects an offline voting-machine workflow, an append-only Solidity bulletin board, and an independent Python verifier. The design combines exponential ElGamal ciphertexts, disjunctive Chaum-Pedersen proofs for candidate bits, a proof that exactly one candidate was selected, signed receipts and batches, and a three-of-five threshold decryption of the verified aggregate. A receipt is prepared before the voter chooses CONFIRM or TEST; a TEST ballot reveals its opening and is spoiled, whereas a CAST ballot contributes only ciphertexts to the tally. A simulated power-off seal and scratch reinstall make transport constraints explicit, but do not attest physical hardware. We describe the threat model, protocol, attack demonstrations, and measurements from the supplied public run: six CAST ballots, one spoiled ballot, three candidates, and three recorded timing samples per operation. We also give transparent arithmetic for a hypothetical election of 150 million voters across 500,000 machines. The measured prototype cannot instantiate that scenario, and the arithmetic is not a deployment or readiness claim. The contribution is an integration and operational design proposal using established cryptographic techniques, not a new primitive or security theorem.

**Keywords:** End-to-end verifiability · electronic voting · threshold cryptography · election audit

## 1 Introduction

An election must both protect a voter's choice and provide a basis for checking the result. These goals do not follow merely from a machine displaying a confirmation or from a ledger recording bytes. A compromised machine can encrypt a different choice, withhold a ballot, or publish a misleading tally; a correct-looking software trace is not proof that the intended device ran. At the same time, a verifiable cryptographic transcript does not establish voter eligibility, prevent coercion, or make an election contestable in law.

Urna Aberta is a compact research prototype organized around three components: an offline urna, a Solidity append-only bulletin board, and an independent Python verifier. The voter-facing sequence is deliberately simple: prepare and

print a receipt, then choose CONFIRM or TEST. A TEST reveals an opening and permanently spoils the ballot. The receipt precedes this choice so that a malicious machine cannot know whether a particular ballot will be checked when it prepares the ciphertext. A confirmed ballot is not published with its encryption randomness. After poll close, machine batches are published before a separately configured upload cutoff; only after that batch deadline does the board close and establish a snapshot. Trustee partial decryptions follow the snapshot and precede a distinct partial deadline.

The system aims to make supplied ballot proofs, batch signatures, receipt inclusion, partial decryptions, and tally arithmetic independently checkable. It does not claim that every current election is unverifiable, or that a blockchain eliminates institutional trust. The manifest and trusted dealer anchor setup, machine and host may be compromised, trustees may collude, and copied JSON is not proof of canonical chain history. The threat model therefore includes adversarial election authorities while making clear that trusted setup anchors remain assumptions.

This paper contributes an integration of established cryptographic mechanisms into a small operational protocol and a reproducible account of its limits. The simulated power seal and scratch-clean-install flow are proposed constraints for a real operation, not new cryptographic guarantees. Our evaluation uses only the existing measurements artifact; it is a small demonstration, not an independent benchmark campaign.

## 2 Threat Model

### 2.1 Assets, adversaries, and goals

Assets include the election manifest and public key, trustee shares, voter choices and encryption randomness before disclosure, the integrity and inclusion of CAST ballots, completeness of the public board export, and the correctness of the announced tally. Adversaries may control a voting machine, alter or omit transport data, register no identity or eligibility, attempt duplicate or rogue-machine submissions, announce false totals, submit malformed proofs, censor a publication, control a local chain, or collude among trustees. A voter may be coerced or lack a private environment. An authority can be adversarial, including an election authority able to direct machines or manipulate an export; the prototype cannot establish that supplied evidence is complete or canonical without an external trust anchor.

The protocol's cryptographic checks target a narrower statement: given an authentic manifest and complete board evidence, the verifier can reject invalid ballot proofs, inconsistent openings, invalid signatures, malformed trustee proofs, or totals inconsistent with the verified aggregate. It cannot infer that the manifest itself is legitimate, that every eligible voter could vote, or that unrecorded ballots do not exist.

### 2.2 Trust anchors and explicit exclusions

A trusted central dealer distributes a three-of-five Shamir sharing; there is no distributed key generation (DKG). The demo process holds all five shares. Any three trustees can decrypt arbitrary ciphertexts. Restricting the public API to an audited aggregate is a policy, not a cryptographic restriction on colluding trustees. The verifier accepts a trusted manifest and an honest, complete board export. Copied JSON is not a consensus proof, light client, or proof of data availability. Anvil can be reset by its operator, and multiple nodes under one operator are not independent institutions.

The physical switch, seal, power-off transport, scratch cleanup, reinstall, and hash comparison are simulations. They do not provide tamper-resistant hardware, secure boot, operating-system attestation, secure erasure, or proof that the approved executable ran. Python is not constant-time, and malicious software may log choices, secrets, or randomness. A signed PREPARED receipt proves neither a physical button press nor voter intent. Its short 20-hex-character code is an 80-bit lookup locator, not a binding proof; verification requires the complete ciphertext-vector hash and machine-bound signature.

There is no voter identity, eligibility, turnout reconciliation, ballot-stuffing protection, or one-person-one-vote mechanism. TEST discloses the selected choice and encryption randomness. Challenge uptake, unpredictability, coercion, and denial of a private voting environment remain open. The contract checks registration, deadlines, signatures, and append rules but does not verify ballot zero-knowledge proofs, partial-decryption proofs, or tally arithmetic on chain. An invalid submitted proof halts off-chain verification. An invalid first aggregate pin can deny service. The 16-machine, 16,000-tally, and 16-MiB event-payload caps, gas, calldata, availability, and block limits prevent national-scale readiness claims.

## 3 Protocol

Let the cyclic subgroup have RFC 3526 group 14 prime $p$ of 2048 bits, order $q = (p-1)/2$, and generator $g = 2$ [7]. Secret exponents are in $\mathbb{Z}_q$; public key $y = g^x$. For message bit $m$ and fresh random $r$, exponential ElGamal is

$$\mathrm{Enc}_y(m; r) = (a, b) = (g^r, y^r g^m) \pmod{p}. \tag{1}$$

The ballot contains one ciphertext per candidate. The bit proof is a Fiat-Shamir OR proof that each ciphertext encrypts either zero or one. Its challenge binds a domain separator, election identifier, public key, the entire ordered ciphertext vector, and candidate index. Thus proofs cannot be moved to another election, key, ballot vector, or position. The sum proof shows equality of discrete logarithms between $(g, A)$ and $(y, B/g)$, where $A = \prod_i a_i$ and $B = \prod_i b_i$; it certifies that the sum of all candidate bits equals one. The prover knows the sum randomness witness $R = \sum_i r_i \bmod q$. In notation, the equality statement is

$$\log_g A = \log_y (B/g) = R \pmod{q}. \tag{2}$$

The exact transcript uses the protocol's canonical JSON, SHA-256, and challenge modulo $q$. Subgroup membership, scalar ranges, canonical encodings, and proof equations are checked before accepting a ballot.

### 3.1 Preparation and voter choice

The machine encrypts a candidate vector and prepares a machine-signed receipt before the voter chooses CONFIRM or TEST. The receipt binds the election, chain, board, machine account, full SHA-256 commitment, and signature domain. The commitment is SHA-256 over the canonical ordered ciphertext vector only; randomized proof bytes are excluded. The compact 20-hex locator is only for lookup, has 80 bits, and is not unique-proof material. The complete digest and signature must be verified. A copied receipt JSON is not itself consensus evidence.

On TEST, the machine reveals the choice and randomness. The verifier recomputes the ciphertexts, checks the opening and the proofs, and marks the ballot SPOILED. On CONFIRM, the randomness is not disclosed and the ciphertext ballot is marked CAST. The signed receipt is not an attestation to a physical button press, nor a mechanism for eligibility or receipt-freeness.

### 3.2 Sealed transport and publication

The operational proposal closes local polling first. A poll worker, rather than the voting application, breaks the simulated seal after close and enables the simulated transport path; software cannot enable it early. The proposed network module is powered off during voting; the simulator models an enable permission, not an actual power circuit. It publishes its complete signed batch before the batch-upload cutoff. The cutoff is distinct from local poll close: this avoids requiring all machines to finish local closing before any machine can upload. Once the batch deadline passes, the board closes and a snapshot is taken; trustee partials are submitted only after that snapshot and before the separate partial deadline. The demo's fixed schedule and contract deadlines should not be conflated with local poll-close events.

Each batch is signed by the same secp256k1 account that sends the transaction. Its EIP-191-signed, domain-separated digest binds chain, board, election, and payload. The contract checks registered signer, deadline, and one-batch rule, then emits payload bytes and a hash. The verifier checks the complete event and signature. A fallback carrier is proposed to copy exactly the same signed bytes, never to regenerate a batch. Scratch reinstallation and hash checks simulate a maintenance step only; neither protects user data nor proves a physical wipe.

### 3.3 Aggregate threshold decryption

The verifier aggregates only CAST ballots after checking the closed board snapshot. Spoiled ballots are excluded. For a candidate ciphertext product $(A, B)$

and trustee share $s_i$, a trustee posts $D_i = A^{s_i}$ with a Chaum-Pedersen proof that $\log_g VK_i = \log_A D_i$, where $VK_i = g^{s_i}$. Each proof binds the election, aggregate digest, trustee index, candidate, key, ciphertext, and commitments. Every proof and registration is checked before interpolation. For three distinct trustee indices, the Lagrange coefficient at zero is

$$\lambda_i = \prod_{\substack{j \in S \\ j \neq i}} \frac{-j}{i-j} \pmod{q}, \qquad D = \prod_{i \in S} D_i^{\lambda_i}. \tag{3}$$

The plaintext group element is $B/D = g^T$, and the verifier solves a bounded discrete logarithm for $T$ within the declared tally range. This yields a public aggregate tally, not individual ballot decryption protection against three colluding trustees.

## 4 Security Analysis

The following cases map to the eight executable attack demonstrations; they illustrate detection paths, not exhaustive guarantees.

### 4.1 Machine changes a choice

A machine may encrypt a choice other than the voter's intended choice. A TEST opening inconsistent with the committed ciphertext is rejected by the verifier, as exercised by `attack-machine`. The test only detects the tested case. A machine can log secrets, target untested ballots, manipulate voter interaction, or exploit low challenge participation. The receipt is printed before the decision to reduce adaptation to an individual challenge, but does not attest to a physical action.

### 4.2 Transport tampering

Editing signed batch bytes changes the payload digest and invalidates the machine signature; the actual contract also rejects the altered submission in `attack-transport`. This proves behavior of a local contract execution under the demonstration setup, not availability, consensus, or resistance to a compromised signing key. Exact-byte carrier fallback is an operational proposal, not cryptographic protection against a malicious machine that signs an incomplete batch.

### 4.3 Dropped ballot

For `attack-drop-vote`, a voter's retained receipt is compared with the signed batch. An absent receipt exposes the omitted ballot when that receipt is supplied. Without an independently retained receipt, an omitted cast may leave no evidence. This is not a completeness proof for turnout or eligibility; the prototype has no identity reconciliation.

### 4.4 False tally

The independent verifier recomputes the result from CAST ballots and verified threshold shares, rejecting announced totals that disagree in `attack-tally`. The verifier uses public evidence, not the demo's private expected selections.

### 4.5 Double vote

Per-candidate OR proofs establish bitness; a single sum equality proof enforces exactly one one-bit. The `attack-double-vote` case demonstrates rejection of a vector with two one-bits. Neither proof prevents stuffing multiple valid ballots or proves a voter is eligible.

### 4.6 Duplicate batch

The Solidity board rejects a second batch from a registered machine in `attack-duplicate`. This check is exercised while the upload window remains open, so expiry is not credited as duplicate detection.

### 4.7 Rogue machine

The board rejects a signed batch from an unregistered sender in `attack-rogue-machine`. These are contract checks against the local deployed code. Trustworthy machine registration and manifest authenticity remain external anchors; an administrator that controls setup can register malicious keys.

### 4.8 Early network access

The `attack-early-network` demonstration shows the application cannot activate its simulated transport before close. It is a software state-machine property, not physical isolation. The power switch and seal are not tamper resistant; physical inspection and institutional custody would be required to give the proposal operational meaning.

### 4.9 Challenge probability

For independent, unpredictable TEST selection with per-ballot probability $p$, the analytic probability of detecting at least one of $k$ altered ballots is $1 - (1 - p)^k$. Figure 1 plots this expression, not observed outcomes. It assumes choices made after commitment and independent of the attacker's knowledge. It does not model strategic voters, coercion, correlated decisions, selection bias, adaptive malware, or low participation. It is not an empirical protection rate or a guarantee.

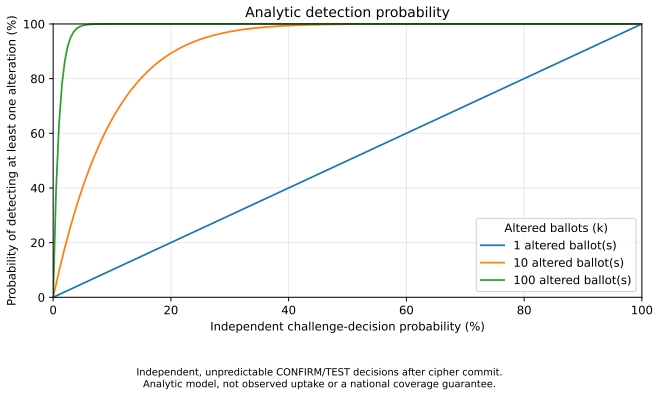



**Fig. 1.** Analytic challenge probability $1 - (1 - p)^k$ under independent, unpredictable TEST choices. This is an equation-based illustration, not measured empirical detection.

### 4.10 Board, trustee, and availability failures

The board does not validate cryptographic ballot or trustee proofs. It pins an aggregate digest, but a malicious first pin can prevent matching honest partials. Three trustees can decrypt arbitrary ballots, whatever the aggregate-only public API policy says. A local chain can be reset, censored, or made unavailable; copied event JSON lacks a proof of canonical history. Physical voter-verifiable records and risk-limiting audits remain necessary for evidence about the paper outcome, and cryptographic receipts are not a substitute for those records.

## 5 Implementation and Evaluation

The prototype has three logical components: an offline Python urna, a Solidity append-only board, and an independent Python verifier. The cryptographic library uses the RFC group and canonical typed records; the board's remit is publication integrity rather than on-chain zero-knowledge verification. The demonstration uses a local Anvil chain. The existing raw JSON records provide measurements and environment metadata; all values below refer to that artifact and its one run, not a new benchmark. Tool installation and first Docker image download/build are excluded. The module boundary is intentionally auditable: canonical typed records are serialized before signature and hash checks, the machine returns its prepared signed batch once closed, and the independent verifier reconstructs the closed-board aggregate before trustees produce a share. This decomposition separates the voter-facing state machine from event publication and mathematical validation, but does not make the components institutionally independent. A single operator can control the local execution and its exported evidence.

**Table 1.** Pure-operation timings from three samples per operation. Values are median and min-max range, in seconds.

| Operation | Median | Range |
|---|---|---|
| Full three-candidate encryption vector | 0.151508750 | 0.151100-0.151819 |
| Bit-proof vector incl. setup and opening checks | 1.088121958 | 1.085648-1.126727 |
| Full ballot incl. all proofs and sum proof | 1.137149792 | 1.134560-1.209563 |
| Public ballot verification | 0.827160750 | 0.823534-0.833354 |

**Table 2.** Recorded batch payload and gas measurements. The first batch is mixed; the second contains three CAST ballots. Gas is a chain execution unit, not a currency fee.

| Batch contents | Payload bytes | Gas used | CAST ballots |
|---|---|---|---|
| First machine, mixed CAST/SPOILED | 44,416 | 1,802,170 | 3 |
| Second machine, CAST only | 32,217 | 1,314,280 | 3 |

The implementation checks canonical records, scalar ranges, subgroup membership, the vector-bound OR challenge and the exactly-one relation described in Section 3. Machine signatures bind the payload to its election, chain and board. These checks prevent malformed data and cross-context reuse; they do not establish the legitimacy of the setup or voter eligibility.

The run contained six CAST ballots, one SPOILED ballot, three candidates, two machines, two batches, and three partial decryptions. CPU architecture is recorded as arm, Python 3.12.11, Anvil 1.8.5, and Darwin 27.2.0. Each pure-operation metric has three samples. The full-demo timings are three fresh process/node samples: 56.590681917 s, 56.174527333 s, and 56.164920292 s. The first recorded process/node is cold; this means first recorded, not flushed CPU or operating-system caches. Later samples are fresh processes and nodes with warmed caches. The separate public verifier ran concurrently in three isolated fresh processes and took 17.819719167 s, 17.830205542 s, and 17.814804209 s.

Bit-proof timing includes ciphertext setup and opening checks; full ballot generation includes all bit proofs and the sum proof, and does not isolate sum-only cost. Figure 2 shows medians and observed ranges from the same three samples, not confidence intervals. The full-demo duration includes more than pure cryptographic operations, while public verifier samples were concurrent; neither is directly interchangeable with a per-ballot sequential estimate.

The published batches contain 76,633 payload bytes in total, including one spoiled ballot and headers; the recorded proof bytes in those batches total 50,974. Each canonical CAST record is 10,622 bytes in the supplied measurement record. Three partial-decryption submissions each used 219,600 gas. Batch transaction receipts are 1,802,170 and 1,314,280 gas. These figures describe a local Anvil run, not a public chain fee or a fixed gas schedule for other payloads. The CAST-only batch of three consumed 1,314,280 gas; the mixed batch of three CAST plus one spoiled consumed 1,802,170 gas.

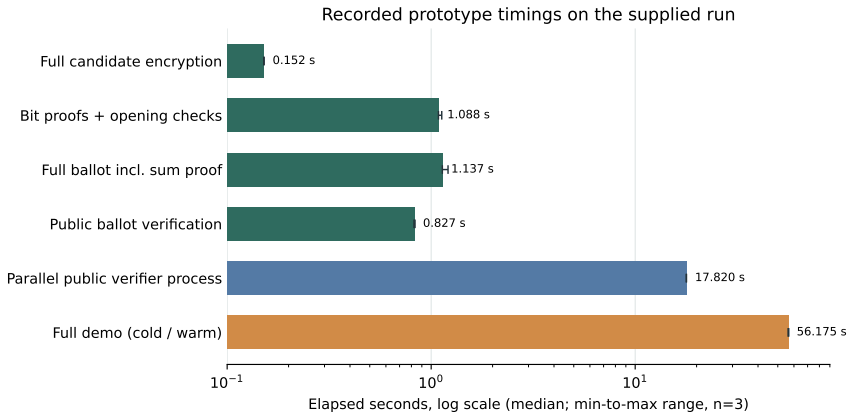



**Fig. 2.** Recorded sample medians and min-to-max ranges from the supplied measurement artifact. These are observations from three samples, not confidence intervals.

### 5.1 Explicit scale arithmetic

Consider only an arithmetic scenario of 500,000 machines and 150,000,000 voters. Dividing gives 300 voters per machine. If every voter contributes one CAST record of the measured 10,622 bytes, and excluding headers, spoiled ballots, replication, and all other overhead, the raw ballot data is $10{,}622 \times 150{,}000{,}000 = 1.5933$ decimal TB. At 300 records per machine, that is 3.1866 MB per device. Actual measured published payload includes headers and a spoiled ballot, so these simplified products are not an estimate of total delivery volume.

A further counterfactual applies the observed gas for one three-CAST batch linearly: 100 such fixed-cost batches imply 131,428,000 gas per 300 CAST ballots and $6.5714 \times 10^{13}$ gas across 150 million voters. This is arithmetic that repeats the measured batch fixed cost, not a large-batch test, network throughput estimate, currency fee, or feasible national deployment. It ignores varying calldata, gas schedules, block limits, failures, and the prototype's 16-machine, 16,000-tally, and 16-MiB payload caps. The implementation cannot instantiate 500,000 machines.

Multiplying the measured public ballot-verification median by 150 million gives 124,074,112.5 sequential core-seconds, about 3.93 core-years. This is a constant-per-ballot arithmetic illustration only. It is not the cost of a complete national verifier: setup, aggregate, proof parsing, I/O, concurrency, hardware, and varying workload are not modeled. It does not show that a home computer can verify a national election quickly. The experiment used three candidates, small data, and three samples, with no confidence intervals or broader hardware sample. The raw artifact records the environment, monotonic samples and receipt provenance. Three observations justify medians and ranges, not confidence intervals or predictions for other processors. Concurrent public-verifier samples

are distinguished from the serial pure-operation and fresh-node full-demo samples.

No architecture for partitioning or sharding is implemented.

## 6 Related Work

Helios established a web-based open-audit voting design for a low-coercion setting and uses a mixnet; it provides useful context for verifiable ballots, but its remote setting does not resolve coercion or software independence [1]. Benaloh's simple verifiable elections describes foundational ballot verification techniques that inform the broader family of voter-checkable encryption systems [2]. ElectionGuard's version 2.1.0 design specification is a contemporary, detailed system specification, but its 4096-bit/256-bit baseline and DPP encryption differ from this RFC-group prototype; Urna Aberta does not claim interoperability, compliance, or equivalent assurance [3].

Scantegrity II integrates end-to-end verification into optical scan elections and retains a paper-based system context absent from this prototype [4]. vVote describes a verifiable system for an election context with mechanisms and assumptions not replicated here; the cited source is its verified version 4 preprint [6]. The underlying equality-of-discrete-log proof follows the Chaum-Pedersen construction [5]; threshold sharing follows Shamir's polynomial secret sharing [10]. These are established techniques, not contributions of this work. Compared with these systems, Urna Aberta deliberately narrows its implementation boundary rather than proposing a replacement election architecture. The use of ciphertext vectors and public proof checking exposes ballot-level validity in a small codebase, while the separate board makes the transport and deadline interface inspectable. Those choices trade deployment completeness for a compact educational example: there is no voter credential layer, mixnet, paper confirmation-code process, or comprehensive institutional protocol. The differences should not be read as a claim that the prototype is simpler to secure or more suitable for actual elections. The chosen group, local ledger and simulated custody differ from those systems and leave deployment obligations unresolved.

Park et al. analyze how Internet voting and blockchain voting can amplify rather than remove security risks, including malware, coercion, contestability, and availability [9]. This work does not answer those criticisms by placing events on a chain: its offline polling-place workflow narrows one exposure but leaves machine compromise, coercion, censorship, and legal contestability unresolved. A ledger is not independent verification by itself. Risk-limiting audits provide a complementary evidence path based on trustworthy voter-verifiable physical records; a ciphertext receipt cannot replace those records [8].

## 7 Limitations

The prototype is educational, with no production, legal-adoption, national-readiness, privacy, or formal-security claim. Setup depends on a trusted central

dealer; there is no DKG. Three trustees can decrypt arbitrary ciphertexts, and the demo process holds five shares. The API's aggregate-only rule is not a cryptographic defense against collusion. Python is not constant-time; software may log choices, shares, or randomness. The simulated switch, seal, transport power-off, cleanup, and reinstall do not demonstrate real physical controls, secure boot, OS attestation, or secure erasure. A PREPARED signature does not prove a button press, and the short receipt locator is not a binding proof.

No eligibility, turnout reconciliation, ballot-stuffing control, or one-person-one-vote mechanism exists. TEST reveals an opening; challenge privacy and uptake remain operational issues. The board does not verify ballot ZK proofs, trustee proofs, or tally arithmetic. A copied JSON board export is not consensus proof, and Anvil is resettable; there is no independent consortium. An invalid proof halts verification, and an invalid first aggregate pin can cause denial of service. Gas, calldata, event-size, availability, censoring, and block limits constrain operation. The existing implementation has hard caps of 16 machines, a tally of 16,000, and 16 MiB of payload; no partitioning or sharding is implemented. These limitations are fundamental to interpreting the small measured workload and the explicitly counterfactual arithmetic.

Cryptographic consistency does not resolve election procedure. A compromised machine can sign an incomplete batch before publication; detection then depends on voters supplying retained receipts. The simulated seal cannot establish custody, exclude alternate radios, or attest installed code. Actual deployment would require independent hardware inspection, accessibility and voter-comprehension evaluation, operational rehearsals, and a legally meaningful dispute process. Voter-verifiable physical records offer independent recount and risk-limiting-audit evidence absent from this prototype. Likewise, the ledger cannot compel inclusion or data availability; its snapshot is not proof that every institution accepted one history. These are excluded obligations, not properties demonstrated by our tests.

## 8 Conclusion

Urna Aberta demonstrates a compact integration of offline ballot preparation, proof-carrying ciphertexts, signed batch publication, and aggregate threshold decryption with a separate verifier. Its receipts and attack cases make several integrity checks inspectable, while its operational proposal separates local poll close, batch publication cutoff, and the later closed-board snapshot. The measurements and scale arithmetic are intentionally bounded by actual artifacts and explicit assumptions. They do not establish deployment readiness, physical security, independent institutional consensus, or universal verifiability of current elections. A realistic election would still need trustworthy physical records and risk-limiting audits, robust governance, independent infrastructure, and careful assessment of coercion and availability.

## Availability and attribution

Source code and measurement artifacts are available in the Urna Aberta repository under Apache License 2.0. This manuscript is authored collectively by Urna Aberta contributors; no institutional affiliation or publication venue is claimed. The bundled Springer LNCS class and bibliography style are separate third-party materials, Springer copyright 1996-2025, licensed under CC BY 4.0 or later. Their inclusion does not change the repository's code license.

## References

1. Adida, B.: Helios: Web-based open-audit voting. In: 17th USENIX Security Symposium (2008), https://www.usenix.org/legacy/events/sec08/tech/full_papers/adida/adida.pdf
2. Benaloh, J.: Simple verifiable elections. In: Proceedings of the USENIX/Accurate Electronic Voting Technology Workshop (2006), https://www.microsoft.com/en-us/research/publication/simple-verifiable-elections/
3. Benaloh, J., Naehrig, M., Pereira, O.: Electionguard design specification. Technical specification Version 2.1.0, Microsoft Research (Aug 2024), https://www.electionguard.vote/spec/
4. Chaum, D., Carback, R., Clark, J., Essex, A., Popoveniuc, S., Rivest, R.L., Ryan, P.Y.A., Shen, E., Sherman, A.T.: Scantegrity ii: End-to-end verifiability for optical scan election systems using invisible ink confirmation codes. In: Proceedings of the USENIX/ACCURATE Electronic Voting Technology Workshop (2008), https://www.usenix.org/legacy/event/evt08/tech/full_papers/chaum/chaum.pdf
5. Chaum, D., Pedersen, T.P.: Wallet databases with observers. In: Advances in Cryptology, CRYPTO '92. Lecture Notes in Computer Science, vol. 740. Springer (1993). https://doi.org/10.1007/3-540-48071-4_7
6. Culnane, C., Ryan, P.Y.A., Schneider, S., Teague, V.: vvote: a verifiable voting system. arXiv:1404.6822v4 (2015). https://doi.org/10.48550/arXiv.1404.6822, https://arxiv.org/abs/1404.6822
7. Kivinen, T., Kojo, M.J.: More modular exponential (modp) diffie-hellman groups for internet key exchange (ike). RFC 3526 (2003). https://doi.org/10.17487/RFC3526, https://www.rfc-editor.org/rfc/rfc3526.txt
8. Lindeman, M., Stark, P.B.: A gentle introduction to risk-limiting audits. IEEE Security and Privacy **10**(5) (2012). https://doi.org/10.1109/MSP.2012.56
9. Park, S., Specter, M., Narula, N., Rivest, R.L.: Going from bad to worse: from internet voting to blockchain voting. Journal of Cybersecurity **7**(1) (2021). https://doi.org/10.1093/cybsec/tyaa025
10. Shamir, A.: How to share a secret. Communications of the ACM **22**(11) (1979). https://doi.org/10.1145/359168.359176